



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 1

Aim: To Implement Loading the data from CSV or JSON files using SQL command (PostgreSQL) or LOAD DATA (MySQL).

Prerequisite: PostgreSQL, MySQL and SQL Commands.

Theory: Fundamentals with Introduction to Data Engineering, Data Extraction, Transformation, and Loading.

Introduction to Data Engineering

Data engineering is the process of designing, building, and maintaining systems that allow organizations to collect, store, process, and analyze large volumes of data. It involves a range of activities such as data pipeline creation, database management, data integration, and optimization to ensure that data is available and accessible for analysis. Data engineers play a crucial role in transforming raw data into a format that is suitable for analysis by data scientists, business analysts, and decision-makers. The field combines knowledge of programming, databases, cloud services, and distributed systems to handle data at scale.

Data Extraction

Data extraction is the first step in the process of obtaining valuable insights from various data sources. It involves retrieving data from different systems, formats, or storage platforms, such as databases, APIs, files, or web scraping. The goal is to extract raw data in a structured or unstructured form, making it ready for processing. Techniques for data extraction vary depending on the source and can involve SQL queries, APIs, or specialized tools like ETL (Extract, Transform, Load) software. The quality and completeness of the extracted data are essential for accurate analysis and decision-making.

Data Transformation

Data transformation is the process of converting raw data into a more usable and meaningful format for analysis. It involves cleaning, filtering, and modifying data to ensure it is consistent, accurate, and structured according to the needs of the end-user or system. This may include tasks like normalizing values, aggregating data, changing data types, handling missing values, and creating new calculated columns. Data transformation is typically performed as part of an ETL process and is crucial for ensuring that data can be effectively analyzed and integrated into business processes.

Data Loading

Data loading is the final step in the ETL (Extract, Transform, Load) pipeline, where the transformed data is loaded into a destination system, such as a data warehouse, database, or analytics platform. This step ensures that the data is stored in a format optimized for querying, reporting, and analysis. The loading process can be either batch-based (loading data in large chunks at scheduled intervals) or real-time (loading data continuously as it becomes available). The choice of data loading method depends on the business needs, the volume of data, and the frequency of updates required. Efficient data loading is key to ensuring timely and accurate access to data for decision-makers.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Implementation Steps:

1) Install PostgreSQL

1. PostgreSQL: Loading Data Using SQL Commands

For CSV Files:

You can use the COPY command or the \copy meta-command in PostgreSQL.

Steps:

1. Ensure the CSV file is accessible to the PostgreSQL server.
2. The table structure in the database must match the structure of the CSV file.
3. Use the COPY command as follows:

Example of loading data into a table named 'users'

```
COPY users (id, name, email, age)
FROM '/path/to/your/file.csv'
DELIMITER ','
CSV HEADER;
```

Explanation:

- users: The table where data will be loaded.
- (id, name, email, age): Column names corresponding to the CSV file columns.
- /path/to/your/file.csv: Full path to the CSV file.
- DELIMITER ',': Defines the delimiter in the CSV file (default is a comma).
- CSV HEADER: Indicates the CSV file has a header row.

Alternative: Using \copy in psql:

If you don't have access to the server filesystem, use the \copy command, which works on the client side.

```
\copy users (id, name, email, age)
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

```
FROM '/path/to/your/file.csv'  
DELIMITER ','  
CSV HEADER;
```

For JSON Files:

PostgreSQL has built-in support for JSON data using the json or jsonb data types. However, you may need to preprocess the JSON file or load it using COPY combined with a temporary table.

1. Create a table with a jsonb column:

```
CREATE TABLE json_data (  
    id SERIAL PRIMARY KEY,  
    data jsonb  
);
```

2. Use the COPY command:

```
COPY json_data(data)  
FROM '/path/to/your/file.json';
```

3. You can then extract or transform the JSON data using PostgreSQL's JSON functions.
-

2. MySQL: Loading Data Using LOAD DATA

For CSV Files:

MySQL supports the LOAD DATA INFILE command to load CSV data efficiently.

Steps:

1. Ensure the CSV file is accessible to the MySQL server.
2. The table structure must match the structure of the CSV file.
3. Use the following command:

```
LOAD DATA INFILE '/path/to/your/file.csv'  
INTO TABLE users  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS  
(id, name, email, age);
```

Explanation:

- LOAD DATA INFILE: Loads the data from a file.
- /path/to/your/file.csv: Full path to the CSV file.
- FIELDS TERMINATED BY ',': Specifies the delimiter in the file.
- ENCLOSED BY '"': Specifies the enclosing character for fields (optional).
- LINES TERMINATED BY '\n': Specifies the line-ending character.
- IGNORE 1 ROWS: Skips the header row.
- (id, name, email, age): Maps the file columns to table columns.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Note: If the file is not accessible on the server, you can use LOAD DATA LOCAL INFILE instead:

```
LOAD DATA LOCAL INFILE '/path/to/your/file.csv'  
INTO TABLE users  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

For JSON Files:

MySQL does not natively support loading JSON files into a table like it does for CSV files. You can:

1. Parse the JSON file in your application (e.g., using Python or JavaScript) and insert data into the table using INSERT statements.
2. Alternatively, use MySQL's JSON data type and insert raw JSON.

Example:

```
CREATE TABLE json_data (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  data JSON  
);
```

-- Insert JSON data directly

```
INSERT INTO json_data (data)
```

```
VALUES ('{"name": "John", "email": "john@example.com", "age": 30}');
```

For bulk insertion, preprocess the JSON file into a CSV format or use a custom script.

OUTPUT SCREENSHOTS:

FOR CSV

1. CREATE DATABASE

```
1 • CREATE DATABASE heart_attack_db;
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

CREATE TABLE

```
CREATE TABLE heart_attack (  
    id SERIAL PRIMARY KEY,  
    age INT,  
    sex INT,  
    total_cholesterol FLOAT,  
    ldl FLOAT,  
    hdl FLOAT,  
    systolic_bp FLOAT,  
    diastolic_bp FLOAT,  
    smoking INT,  
    diabetes INT,  
    heart_attack INT  
);
```

CREATE TABLE

Query returned successfully in 128 msec.

2. LOAD CSV FILE

```
COPY heart_attack(age, sex, total_cholesterol, ldl, hdl, systolic_bp, diastolic_bp, smc  
FROM 'C:\Program Files\PostgreSQL\15\data\updated_version.csv'  
DELIMITER ','  
CSV HEADER;
```

COPY 1000

Query returned successfully in 78 msec.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

3. DISPLAY TABLE

```
SELECT * FROM heart_attack;
```

Data Output Messages Notifications												
	id [PK] integer	age integer	sex integer	total_cholesterol double precision	ldl double precision	hdl double precision	systolic_bp double precision	diastolic_bp double precision	smoking integer	diabetes integer	heart_attack integer	
1	1	57	1	229.46364245989878	175.87912925154933	39.22568705527789	124.07012716313945	91.37878025428563	0	0	0	
2	2	58	1	186.46411960718476	128.9849160885468	34.95096762813148	95.49255176716945	64.35504019073372	1	0	0	
3	3	37	1	251.30071919021645	152.34759243501477	45.91328769795121	99.51933468568974	64.95314701865036	0	1	0	
4	4	55	1	192.05890823360733	116.80368391178928	67.20892502595552	122.46000218901882	73.8213817518664	0	0	0	
5	5	53	1	151.20344845928565	107.01739560621428	60.69383790566267	123.02225716752147	81.12194576039188	0	1	0	
6	6	39	1	236.0334551699892	153.88080937296337	31.208614010237696	121.85739560481021	79.58906898043419	0	0	0	
7	7	65	0	174.6156654741166	114.02940746386828	55.69258599001828	135.60504971915043	85.52995490168766	0	0	0	
8	8	33	0	242.91940159950704	147.95137529846954	54.4394748110479	123.51155667862578	77.33171433136184	0	0	0	
9	9	49	0	95.80435914010542	83.30487485574838	60.758928918985845	111.69748795963216	77.63052930816697	1	0	0	
10	10	55	0	181.36094256462488	106.01178251788478	50.57674663030641	129.57641759900724	87.58878129615607	0	0	0	
11	11	51	0	172.48884163155563	108.37382246121102	51.2036260423836	96.82124634895237	69.57381612195707	0	0	0	
12	12	46	1	182.77891098453784	113.6543939042498	46.83824528078549	114.31364253763297	79.97690932376277	0	0	0	
13	13	54	1	126.9359199405608	88.47328560934073	59.03780511574431	151.13849059330406	96.92753428227385	0	0	0	
14	14	33	0	219.195145421094	149.29948851137334	39.257935850175556	135.85762085722814	81.29240087203232	0	0	0	
15	15	35	1	196.43572789632594	115.05486999260103	40.61167128512642	113.22721541605009	71.2237534490105	0	0	0	
16	16	73	1	200.40970895710234	151.5826471964596	35.832455981749455	129.9974758740287	93.53521042451113	1	0	1	
17	17	54	0	152.98228660589035	80.54516819341904	63.81022315315402	119.63183529642467	81.36480595501722	0	0	0	
18	18	53	0	173.63153909414024	148.25682007086246	50.798153909782016	108.55837553210901	77.91457649513478	0	0	0	
19	19	67	1	247.6771509726762	170.19838132056907	48.25316658394561	156.19553877301414	96.90645422915483	0	0	0	
20	20	38	1	206.90306810921524	135.2881048032097	55.74446873727193	119.28299242337216	87.95509922514194	0	0	0	
21	21	59	1	163.73310940575246	120.58903417798795	50.280963120194926	131.69316724202164	89.72471207901188	0	0	0	
22	22	28	1	221.7970108896927	139.90417877475313	38.288142247820076	116.22689116706718	81.21362757245257	1	0	0	
Total rows: 1000 of 1000 Query complete 00:00:00.297												



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

FOR JSON

1. CREATE TABLE

```
1 CREATE TABLE digital_literacy (  
2     user_id TEXT PRIMARY KEY,  
3     age INT,  
4     gender TEXT,  
5     education_level TEXT,  
6     employment_status TEXT,  
7     household_income TEXT,  
8     location_type TEXT,  
9     basic_computer_knowledge_score INT,  
10    internet_usage_score INT,  
11    mobile_literacy_score INT,  
12    post_training_basic_computer_knowledge_score INT,  
13    post_training_internet_usage_score INT,  
14    post_training_mobile_literacy_score INT,  
15    modules_completed INT,  
16    average_time_per_module FLOAT,  
17    quiz_performance INT,  
18    session_count INT,  
19    engagement_level TEXT,  
20    adaptability_score INT,  
21    feedback_rating INT,  
22    skill_application INT,  
23    employment_impact TEXT,  
24    overall_literacy_score FLOAT  
25 );  
26
```

CREATE TABLE

Query returned successfully in 76 msec.

2. INSERT JSON FILE INTO SQL USING PYTHON SCRIPT



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

```
import json.py > ...
1 import json
2 import psycopg2
3
4 # Load JSON file
5 file_path = "F:\digital_literacy_dataset.json" # Adjust this path if needed
6 with open(file_path, "r") as f:
7     data = json.load(f)
8
9 # Connect to PostgreSQL
10 conn = psycopg2.connect(
11     dbname="...",
12     user="...",
13     password="...",
14     host="localhost",
15     port="5432"
16 )
17 cur = conn.cursor()
18
19 # Insert data into the table
20 for row in data:
21     cur.execute("""
22         INSERT INTO digital_literacy (
23             user_id, age, gender, education_level, employment_status, household_income,
24             location_type, basic_computer_knowledge_score, internet_usage_score,
25             mobile_literacy_score, post_training_basic_computer_knowledge_score,
26             post_training_internet_usage_score, post_training_mobile_literacy_score,
27             modules_completed, average_time_per_module, quiz_performance, session_count,
28             engagement_level, adaptability_score, feedback_rating, skill_application,
29             employment_impact, overall_literacy_score
30         ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
31     """, (
32         row["User_ID"], row["Age"], row["Gender"], row["Education_Level"], row["Employment_Status"],
33         row["Household_Income"], row["Location_Type"], row["Basic_Computer_Knowledge_Score"],
```

```
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
34     """, (
35         row["User_ID"], row["Age"], row["Gender"], row["Education_Level"], row["Employment_Status"],
36         row["Household_Income"], row["Location_Type"], row["Basic_Computer_Knowledge_Score"],
37         row["Internet_Usage_Score"], row["Mobile_Literacy_Score"], row["Post_Training_Basic_Computer_Knowledge_Score"],
38         row["Post_Training_Internet_Usage_Score"], row["Post_Training_Mobile_Literacy_Score"],
39         row["Modules_Completed"], row["Average_Time_Per_Module"], row["Quiz_Performance"],
40         row["Session_Count"], row["Engagement_Level"], row["Adaptability_Score"], row["Feedback_Rating"],
41         row["Skill_Application"], row["Employment_Impact"], row["Overall_Literacy_Score"]
42     ))
43
44 # Commit and close connection
45 conn.commit()
46 cur.close()
47 conn.close()
48
49 print("Data imported successfully!")
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

3. DISPLAY TABLE

```
SELECT * FROM digital_literacy LIMIT 10;
```

	user_id [PK] text	age integer	gender text	education_level text	employment_status text	household_income text	location_type text	basic_computer_knowledge_score integer	internet_usage_score integer	mobile_literacy_score integer	post_training_basic_co integer
1	U0001	43	Male	Primary	Student	Medium	Semi-Rural	25	1	33	
2	U0002	60	Female	High School	Farmer	Low	Rural	22	14	35	
3	U0003	47	Female	Primary	Farmer	Low	Semi-Rural	14	31	14	
4	U0004	34	Female	Secondary	Farmer	Low	Rural	6	32	17	
5	U0005	50	Male	High School	Other	Medium	Rural	14	41	19	
6	U0006	57	Female	High School	Self-Employed	Low	Rural	15	3	46	
7	U0007	46	Other	Secondary	Unemployed	Low	Semi-Rural	35	34	49	
8	U0008	36	Male	High School	Unemployed	Low	Rural	19	48	46	
9	U0009	64	Male	High School	Other	Low	Rural	33	10	37	
10	U0010	61	Female	High School	Self-Employed	Medium	Rural	50	18	20	

Conclusion

We successfully loaded JSON and CSV data into a PostgreSQL table using COPY and PYTHON. The process included:

1. **Creating the digital_literacy table** with columns matching your JSON structure.
2. **Inserting the data** while ensuring proper UTF-8 encoding (using utf8mb4).



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

3. **Handling a character set warning** by switching to `utf8mb4` for better compatibility.
4. **Verifying the data** was correctly inserted by running a `SELECT` query.